

Robot Operating Systems

Robot Software Development



Abdelbacet Mhamdi

2025-11-18

MT @ ISET Bizerte

1. ROS 2 Foundations	2
2. Nodes and Communication	41
3. Custom Interface Development	67

1. ROS 2 Foundations

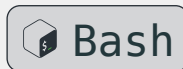
1.1 A Practical Introduction

TurtleSim is a beginner-friendly tool for learning **ROS 2** concepts through visual, interactive examples. This lightweight simulator offers a 2D environment where we control virtual turtles that can move around, draw lines, and respond to commands.

1.1.1 Installation

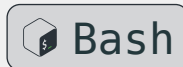
TurtleSim comes pre-installed with the **ROS 2** desktop installation. We can also install it using the following command:

```
1 sudo apt install ros-humble-turtlesim # For Ubuntu 22: ROS 2 Humble
```



The way to verify the installation is by checking the available executables:

```
1 ros2 pkg executables turtlesim
```



1.1 A Practical Introduction

1.1.2 Launching TurtleSim

The first step is launching the *turtlesim_node*, which creates the simulation window:

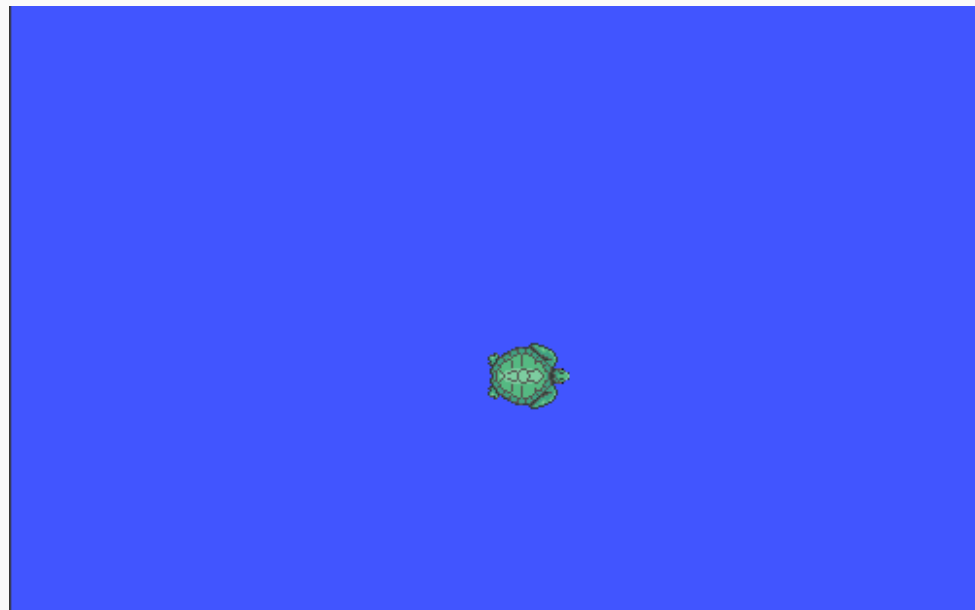
```
1 ros2 run turtlesim turtlesim_node
```



This command starts a blue simulation window with a turtle in the center.

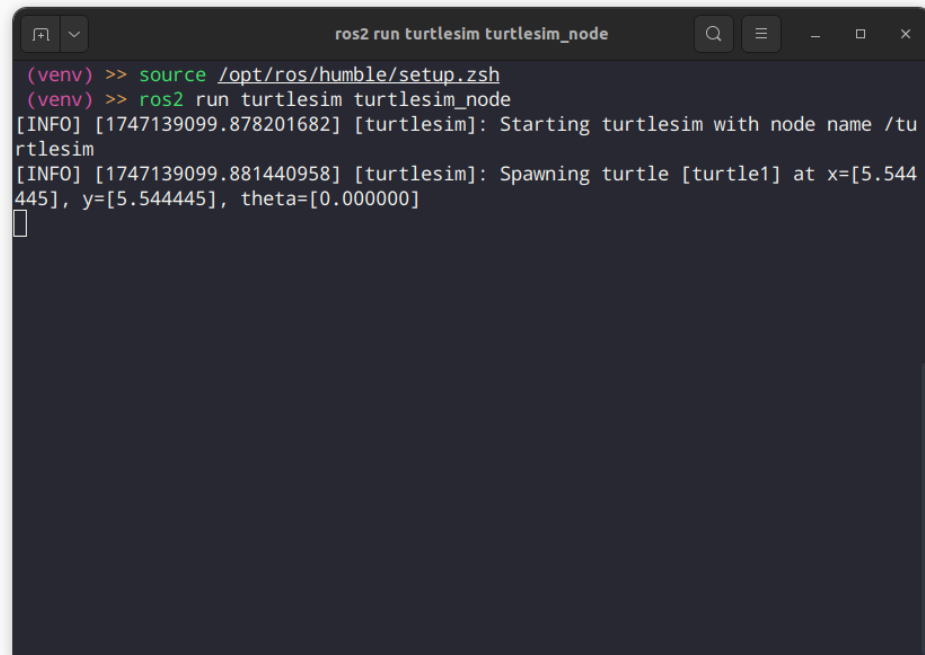
This node:

- creates the simulation environment
- manages turtle state (position, orientation, pen status)
- processes incoming commands
- publishes turtle pose information



1.1 A Practical Introduction

- The turtle's position and orientation are represented in a 2D coordinate system.
- The turtle can move forward, backward, and rotate, and its pen can be raised or lowered to draw on the canvas.



```
ros2 run turtlesim turtlesim_node

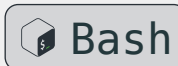
(venv) >> source /opt/ros/humble/setup.zsh
(venv) >> ros2 run turtlesim turtlesim_node
[INFO] [1747139099.878201682] [turtlesim]: Starting turtlesim with node name /turtlesim
[INFO] [1747139099.881440958] [turtlesim]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445], theta=[0.000000]
```

1.1 A Practical Introduction

1.1.2.1 List Active Components

The turtle's state is managed by the *TurtleSim* node, which processes incoming commands and publishes the turtle's pose information. This allows us to control the turtle's movement and visualize its position in real time.

```
1 # running nodes
2 ros2 node list
3 # active topics
4 ros2 topic list
5 # available services
6 ros2 service list
7 # display actions
8 ros2 action list
```

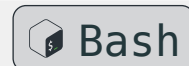


```
mhamdi@ubuntu:~$ ros2 run turtlesim turtlesim_node
(venv) >> ros2 node list
/turtlesim
(venv) >> ros2 topic list
/parameter_events
/rosout
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
(venv) >> ros2 service list
/clear
/kill
/reset
/spawn
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/describe_parameters
/turtlesim/get_parameter_types
/turtlesim/get_parameters
/turtlesim/list_parameters
/turtlesim/set_parameters
/turtlesim/set_parameters_atomically
(venv) >> ros2 action list
/turtle1/rotate_absolute
(venv) >>
```

1.1 A Practical Introduction

1.1.2.2 Moving the Turtle

```
1 ros2 run turtlesim turtle_teleop_key
```



Info

Keep the teleop terminal window focused while sending keyboard commands.

Memorize

Arrow keys Move the turtle forward/backward and rotate left/right

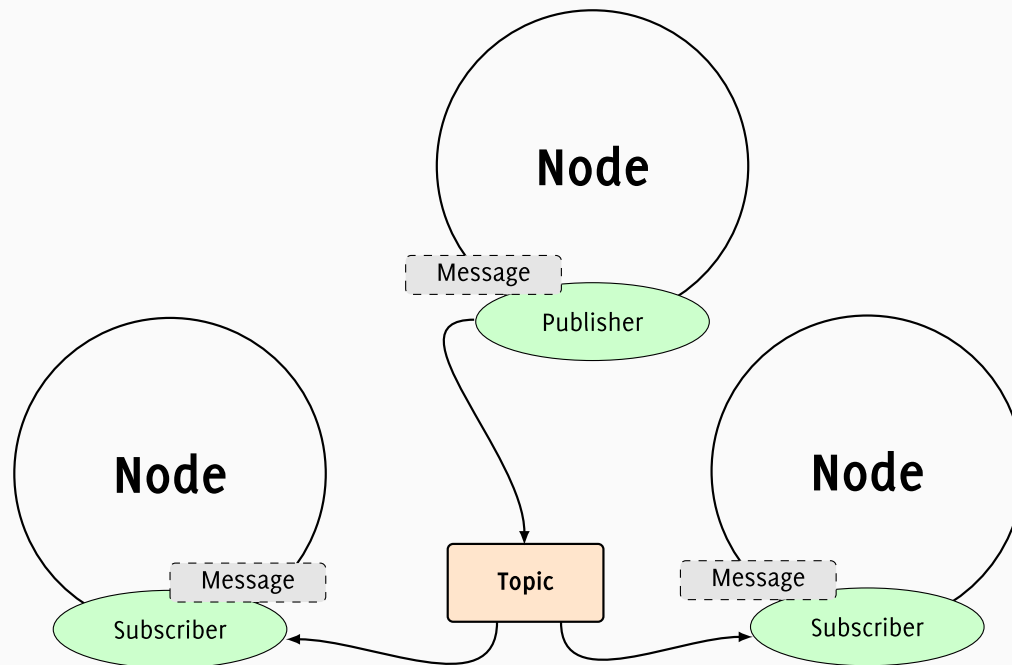
Space Stop the turtle

1.1 A Practical Introduction

1.1.3 Programmatic Control

1.1.3.1 Using Topics

Topics are named communication channels for message passing.



1.1 A Practical Introduction

We can explore active topics via:

```
1 # List all active topics
2 ros2 topic list
```



Key topics include:

/turtle1/cmd_vel For sending velocity commands

/turtle1/color_sensor Color detected by the turtle

/turtle1/pose Published turtle position and orientation

Examine topic types and message structures:

```
1 ros2 topic type /turtle1/cmd_vel # Result: geometry_msgs/msg/Twist
```



```
1 ros2 topic echo /turtle1/pose # Echo messages on a topic
```



1.1 A Practical Introduction

1. Forward

```
1 ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear:
  {x: 2., y: 0., z: 0.}, angular: {x: 0., y: 0., z: 0.}}"
```



2. Backward

```
1 ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear:
  {x: -2., y: 0., z: 0.}, angular: {x: 0., y: 0., z: 0.}}"
```



3. Rotate Left

```
1 ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear:
  {x: 0., y: 0., z: 0.}, angular: {x: 0., y: 0., z: 1.}}"
```



4. Rotate right

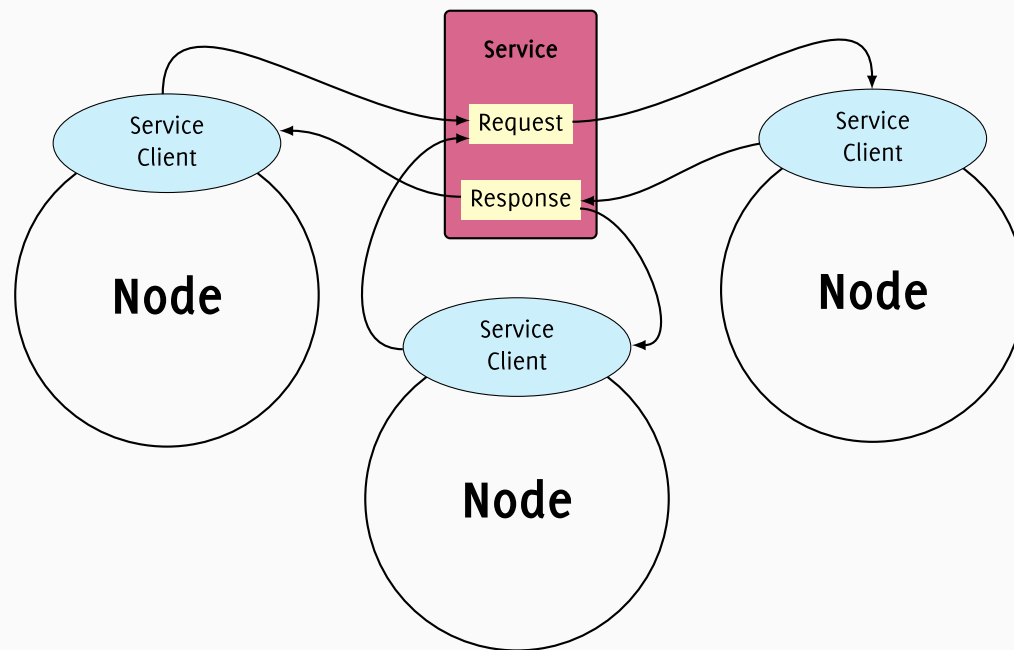
```
1 ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear:
  {x: 0., y: 0., z: 0.}, angular: {x: 0., y: 0., z: -1.}}"
```



1.1 A Practical Introduction

1.1.3.2 Using Services

Services enable us to request specific actions or information from the turtle's services.



1.1 A Practical Introduction

To list all active services, we use:

```
1 ros2 service list -t
```

 Bash

1. Set Absolute Position

```
1 ros2 service call /turtle1/teleport_absolute turtlesim/srv/  
TeleportAbsolute "{x: 5., y: 5., theta: 0.}"
```

 Bash

2. Set Relative Position

```
1 ros2 service call /turtle1/teleport_relative turtlesim/srv/  
TeleportRelative "{linear: 1., angular: 1.}"
```

 Bash

3. Changing the Turtle's Color

```
1 ros2 service call /turtle1/set_pen turtlesim/srv/SetPen "{r: 255,  
g: 0, b: 0, width: 2, 'off': 0}"
```



4. Clearing the Screen

```
1 ros2 service call /clear std_srvs/srv/Empty "{}"
```



5. Resetting the Turtle

```
1 ros2 service call /reset std_srvs/srv/Empty "{}"
```



1.2 Turtle Drawing Challenge

Use `turtlesim` to create a square pattern using only **ROS 2** command-line tools. The goal is to make the turtle draw a perfect square without any programming.

1.2.1 What to Observe

- How does the turtle's movement affect the drawing?
- What happens when you change the pen color mid-drawing?
- How precise can you make the square using only command-line tools?

1.2.2 Learning Objectives

- Understand **ROS 2** topic publishing
- Learn about service calls
- Practice with `geometry_msgs/msg/Twist` messages
- Experience real-time robot control concepts

1.2 Turtle Drawing Challenge

Task 1: Pen Control

Before starting the square, set the pen to:

- Red color (R=255, G=0, B=0)
- Width of 3 pixels
- Pen down (off=0)

1.2 Turtle Drawing Challenge

```
1 # Set pen to red color and width 3
```



```
2 ros2 service call /turtle1/set_pen turtlesim/srv/SetPen "{r: 255, g: 0,  
  b: 0, width: 3, 'off': 0}"
```

1.2 Turtle Drawing Challenge

Task 2: Basic Movement

Use the `ros2 topic pub` command to make the turtle:

- Move forward 2 units
- Rotate 90 degrees to the right
- Move forward 2 units
- Rotate 90 degrees to the right
- Move forward 2 units
- Rotate 90 degrees to the right
- Move forward 2 units
- Rotate 90 degrees to the right

1.2 Turtle Drawing Challenge

Tip

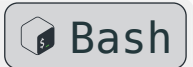
Use geometry_msgs/msg/Twist messages with:

- linear.x for forward/backward movement
- angular.z for rotation (positive = left, negative = right)

Info

Don't forget to reset the turtle's position if needed:

```
1 ros2 service call /reset std_srvs/srv/Empty "{}"
```



1.2 Turtle Drawing Challenge

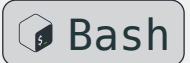
```
1 # Move forward 2 units
2 ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear:
  {x: 2.0}}}"
3
4 # Rotate 90 degrees right
5 ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{angular:
  {z: -1.57}}}"
```



1.2 Turtle Drawing Challenge

We could also create a square using /turtle1/teleport_absolute service:

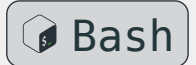
```
1 # Teleport to corners of square
2 ros2 service call /turtle1/teleport_absolute turtlesim/srv/
  TeleportAbsolute "{x: 7.5, y: 5.5}" && sleep 1
3
4 ros2 service call /turtle1/teleport_absolute turtlesim/srv/
  TeleportAbsolute "{x: 7.5, y: 3.5}" && sleep 1
5
6 ros2 service call /turtle1/teleport_absolute turtlesim/srv/
  TeleportAbsolute "{x: 5.5, y: 3.5}" && sleep 1
7
8 ros2 service call /turtle1/teleport_absolute turtlesim/srv/
  TeleportAbsolute "{x: 5.5, y: 5.5}"
```



1.2 Turtle Drawing Challenge

Using /turtle1/teleport_relative service is also possible:

```
1  ros2 service call /turtle1/teleport_relative turtlesim/srv/  
   TeleportRelative "{linear: 2.}" && sleep 1  
2  
3  ros2 service call /turtle1/teleport_relative turtlesim/srv/  
   TeleportRelative "{linear: -2., angular: 1.57}" && sleep 1  
4  
5  ros2 service call /turtle1/teleport_relative turtlesim/srv/  
   TeleportRelative "{linear: 2., angular: 1.57}" && sleep 1  
6  
7  ros2 service call /turtle1/teleport_relative turtlesim/srv/  
   TeleportRelative "{linear: 2., angular: -1.57}"
```



1.2 Turtle Drawing Challenge

Task 3: Precision Challenge

Try to make the turtle return to its starting position after completing the square. Use the `teleport_absolute` service to verify the final position.

1.2 Turtle Drawing Challenge

To check if the turtle returned to its starting position:

```
1 # Check current position
2 ros2 topic echo /turtle1/pose
3
4 # If needed, teleport to exact starting position
5 ros2 service call /turtle1/teleport_absolute turtlesim/srv/
  TeleportAbsolute "{x: 5.5, y: 5.5, theta: 0.0}"
```



1.2 Turtle Drawing Challenge

1.2.3 Summary and Key Takeaways

Understanding Twist Messages

- `linear.x`: Forward/backward velocity (positive = forward)
- `angular.z`: Rotational velocity (positive = counterclockwise, negative = clockwise)
- Values are in radians per second for rotation

Timing Considerations

- The `sleep` commands are crucial for allowing the turtle to complete each movement
- Without proper timing, movements overlap and the pattern becomes distorted
- Real robots would use feedback control instead of fixed timing

Service vs Topic Usage

- **Topics**: For continuous commands (velocity control)
- **Services**: For discrete actions (teleportation, pen control)

1.2 Turtle Drawing Challenge

Warning

- If the turtle doesn't move, check that the Turtlesim window is active
- If movements are too fast/slow, adjust the sleep duration
- If the square isn't closed, verify the rotation angles ($90^\circ = \frac{\pi}{2} \approx 1.57$ radians)
- Use `ros2 topic echo /turtle1/pose` to monitor the turtle's position in real-time

Idea

- Try creating different shapes (triangle, pentagon, hexagon)
- Experiment with different pen colors and widths
- Create patterns by combining multiple shapes
- Use relative teleportation for more complex movements

1.3 What is ROS?

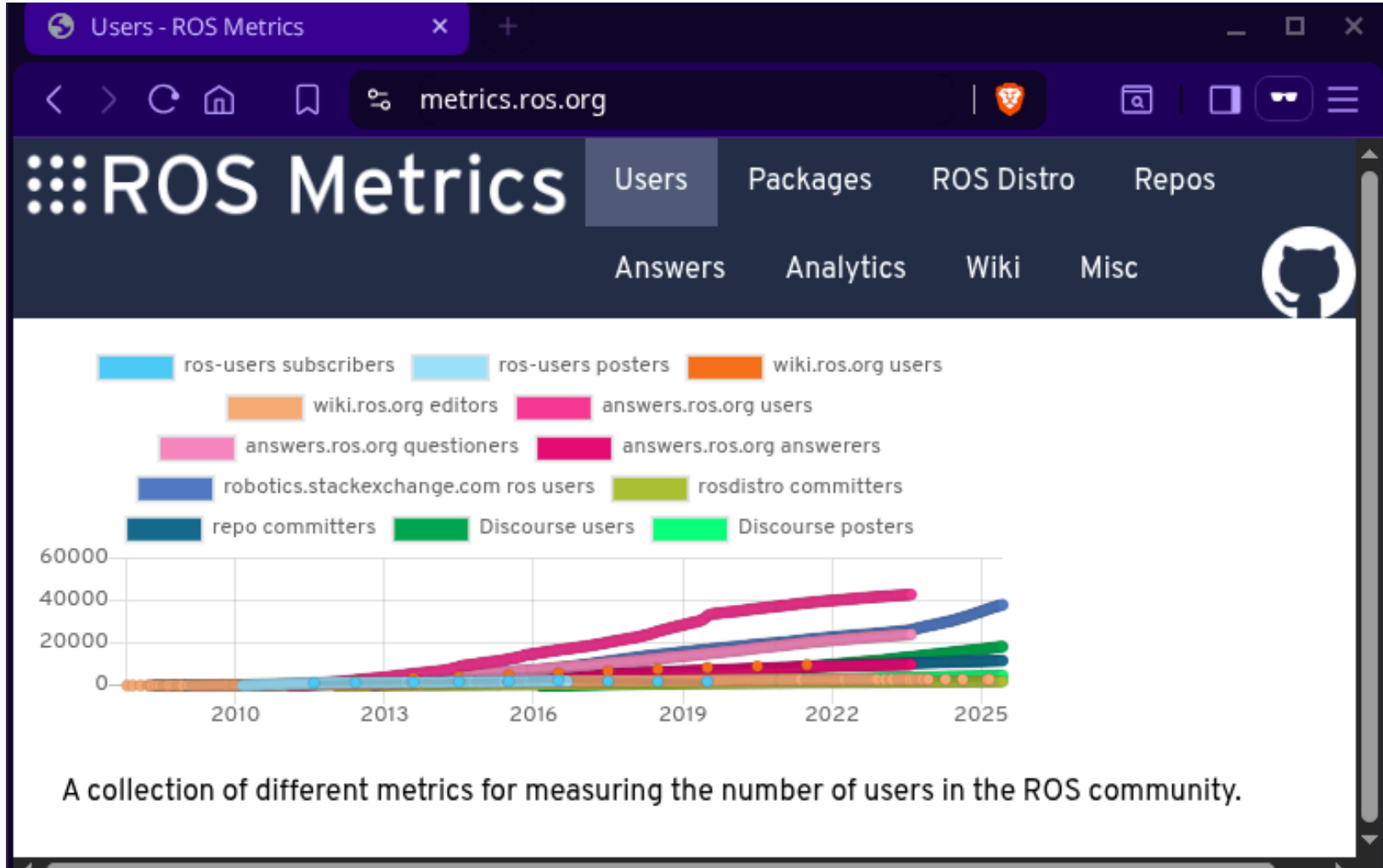
The **Robot Operating System (ROS)** is an open-source, flexible framework for writing robot software. It is not an operating system in the traditional sense but rather a collection of software libraries, tools, and conventions that aim to simplify the task of creating complex and robust robot behavior across a wide variety of robotic platforms.

ROS provides functionalities typically expected from an **OS**, including hardware abstraction, low-level device control, implementation of commonly-used functionality, message-passing between processes, and package management.

Goal

Its primary goal is to support code reuse in robotics research and development.

1.3 What is ROS?



1.3 What is ROS?

Modular Architecture

ROS is built on a distributed computing model, allowing developers to create modular software packages that can easily communicate with each other. This approach enables:

- Flexible robot software design
- Easy integration of different components
- Reusability of code across different robotic projects

1.3 What is ROS?

Communication Infrastructure

At its core, **ROS** provides a robust communication infrastructure that allows different parts of a robotic system to exchange information:

- Publish-subscribe messaging model
- Service-client communication
- Action servers for long-running tasks
- Support for multiple programming languages (primarily C++ and Python)

1.3 What is ROS?

Extensive Toolset

ROS offers a comprehensive set of tools for various aspects of robotics development:

- Simulation environments (like **Gazebo**)
- Visualization tools (like **RViz**)
- Debugging and logging utilities
- Hardware abstraction layers
- Package management system

1.3 What is ROS?

Applications and Usage

ROS is widely adopted in:

- Academic research institutions
- Robotics laboratories
- Industrial robotics
- Autonomous vehicle development
- Research and development in various domains (manufacturing, healthcare, aerospace)

1.3 What is ROS?

⚡ Danger

Limitations of ROS 1

While ROS 1 achieved significant success, the evolving robotics landscape highlighted several limitations in its design, including:

Multi-robot systems Struggled with decentralized communication and discovery in large or dynamic networks.

Real-time control Lacked native support for real-time performance requirements.

Production environments Offered limited security features, making it less suitable for commercial deployments.

Resource-constrained platforms Proved resource-heavy for embedded systems.

Network reliability Depended on a single master node, introducing a single point of failure.

1.3 What is ROS?



Success

ROS 2 Overview

ROS 2 was designed from scratch to overcome the limitations of ROS 1, delivering a more robust, secure, and adaptable platform. It supports a wide range of applications, from research prototypes to industrial and commercial systems.

1.3 What is ROS?

- Enhanced Multi-Robot Support** Enables decentralized communication and discovery, addressing ROS 1's challenges with dynamic, large-scale robot networks.
- Real-Time Capabilities** Provides native support for real-time control, making it suitable for time-critical robotic applications.
- Improved Security** Incorporates robust security features to meet the needs of commercial and industrial deployments.
- Resource Efficiency** Optimized for resource-constrained platforms, such as embedded systems, unlike the heavier ROS 1.
- Network Reliability** Eliminates the single point of failure by removing dependency on a single master node, improving system resilience.

1.3 What is ROS?

Distro	Logo	Release	EOL
Kilted Kaiju		23 mai 2025	December 2026
Jazzy Jalisco		23 mai 2024	May 2029
Iron Irwini		23 mai 2023	November 2024
Humble Hawksbill		23 mai 2022	mai 2027
Galactic Geochelone		23 mai 2021	9 december 2022

1.3 What is ROS?

Foxy Fitzroy		5 juin 2020	20 june 2023
Eloquent Elusor		22 novembre 2019	novembre 2020
Dashing Diademata		31 mai 2019	mai 2021
Crystal Clemmys		14 december 2018	december 2019
Bouncy Bolson		2 july 2018	july 2019
Ardent Apalone		8 december 2017	december 2018

1.3 What is ROS?



Notification

Introducing ROS 2 Humble Hawksbill

ROS 2 Humble Hawksbill, the eighth major release of **ROS 2**, was officially launched on May 23, 2022. It marks a key milestone in the **ROS 2** ecosystem, enhancing previous versions with new features and improvements.

1.3 What is ROS?

- **Humble Hawksbill** is an LTS release, offering stability and extended maintenance for developers.
- Support includes bug fixes and security patches until May 2027.
- Its long support period suits commercial products, extended research projects, and educational use.
- Primarily targets **Ubuntu 22.04 Jammy Jellyfish** (amd64 and arm64 architectures).
- Also compatible with **Windows 10**.

1.3 What is ROS?

Improvements in Humble

- ROSBag Enhancements
- Performance and Stability Gains
- Enhanced Documentation
- Developer Ergonomics
- Gazebo Fortress Integration
- FogROS2
- Foxglove Studio

2. Nodes and Communication

2.1 What is a ROS 2 Node?

A **ROS 2** node is a fundamental computational unit in the Robot Operating System 2 that represents an executable process within a robotic system.

2.1.1 Core Components

- Written in C++ or Python
- Uses `rclcpp` (C++) or `rclpy` (Python) client libraries
- Supports multiple communication patterns

2.1 What is a ROS 2 Node?

2.1.2 Types of Nodes

2.1.2.1 Sensor Nodes

- Collect and process sensor data



Example

- Camera node
- LIDAR node
- IMU sensor node

2.1 What is a ROS 2 Node?

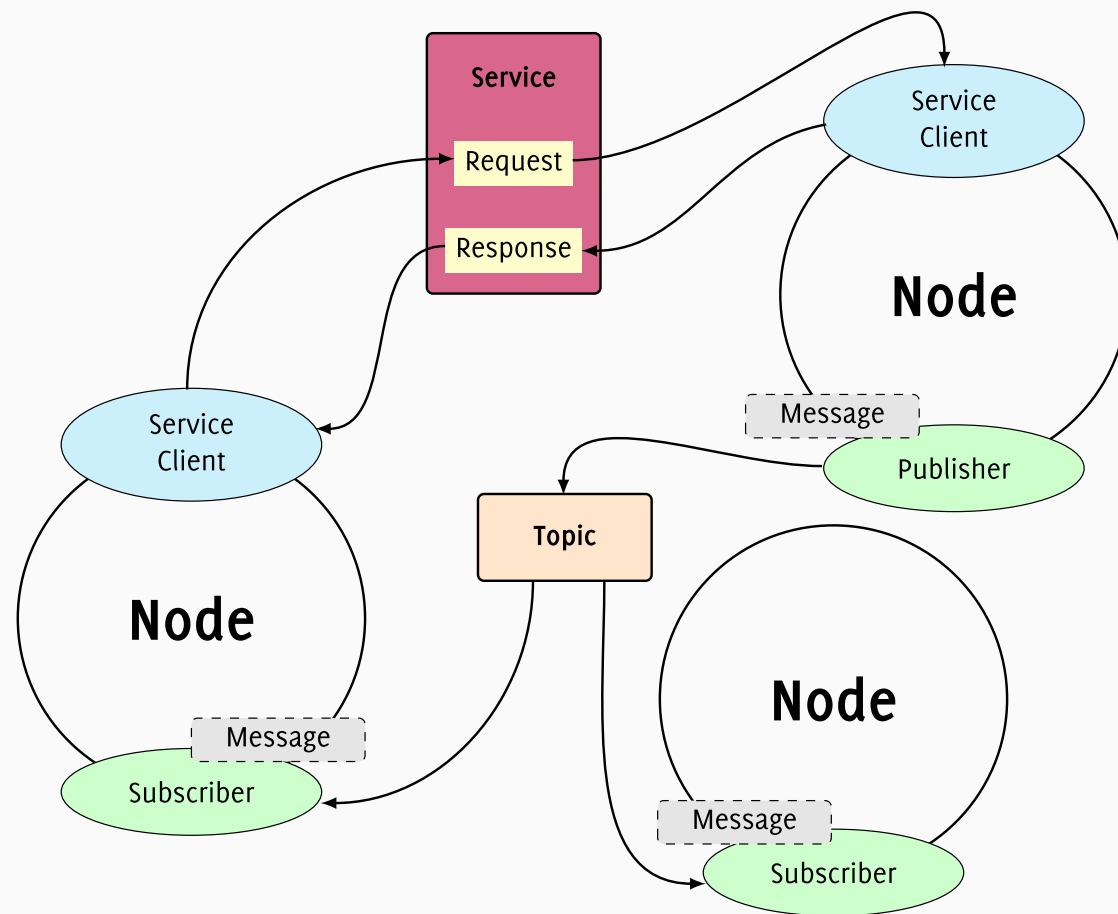
2.1.2.2 Actuator Nodes

- Control robotic actuators
- Manage precise motor movements

2.1.2.3 Processing Nodes

- Implement computational algorithms
- Perform data analysis and decision-making

2.1 What is a ROS 2 Node?



2.1 What is a ROS 2 Node?

```
1 # Launch individual nodes
2 ros2 run <pkg_name> <node_name>
3 # Launch multiple nodes with configuration
4 ros2 launch <pkg_name> <launch_file>
5 # Display active nodes
6 ros2 node list
7 # Show detailed information about a specific node
8 ros2 node info <node_name>
```



2.2 Communication Patterns

Topics, **services**, and **actions** are core communication mechanisms used to enable nodes to exchange data, request computations, or manage long-running tasks.

2.2.1 Topics

- Provide a publish-subscribe communication model
- Allow nodes to send messages to multiple subscribers
- Useful for broadcasting information

```
1 # Publisher
2 ros2 topic pub /chatter std_msgs/msg/String "data: 'Hello, ROS 2!'"
3 # Subscriber
4 ros2 topic echo /chatter
```



2.2 Communication Patterns

2.2.2 Services

- Provide a request-response communication model
- Allow nodes to call functions on other nodes
- Useful for tasks that require a single response

```
1 # Server
2 ros2 run demo_nodes_cpp add_two_ints_server
3 # Client
4 ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 5,
  b: 3}"
```



2.2 Communication Patterns

2.2.3 Actions

- Designed for long-running tasks
- Allow feedback during execution
- Support preemption and cancellation

2.2 Communication Patterns

Task 4: Theory

1. What is a **ROS 2** node?
2. What is the difference between a topic and a service in **ROS 2**?
3. Give an example use case for a topic and one for a service.

2.2 Communication Patterns

1. A **ROS 2** node is a fundamental process that performs computation in a **ROS 2** system. Each node is an independent executable that can communicate with other nodes using topics, services, and actions.
2. A topic in **ROS 2** is used for unidirectional, asynchronous, many-to-many communication (publish/subscribe), suitable for streaming data like sensor readings. A service is used for synchronous, two-way communication (request/response), suitable for tasks that require a reply, like resetting a simulation.
3. **Example use case for a topic** Publishing sensor data from a LIDAR to be consumed by multiple nodes.

Example use case for a service A node requests another node to reset the simulation environment and waits for confirmation.

2.2 Communication Patterns

Task 5: Practical

4. Write the command to list all active nodes in a running **ROS 2** system.
5. Suppose you have a node publishing to the topic `/chatter`. Write the command to echo messages from this topic.
6. What is the command to call a service named `/reset_simulation` of type `std_srvs/srv/Empty`?

2.2 Communication Patterns

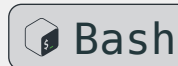
4. Command to list all active nodes:

```
1 ros2 node list
```



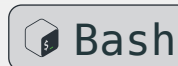
5. Command to echo messages from the /chatter topic:

```
1 ros2 topic echo /chatter
```



6. Command to call the /reset_simulation service of type std_srvs/srv/Empty:

```
1 ros2 service call /reset_simulation std_srvs/srv/Empty "{}"
```



2.3 Build mechanism

```
1 ros2 pkg create --build-type ament_python <package_name>
```

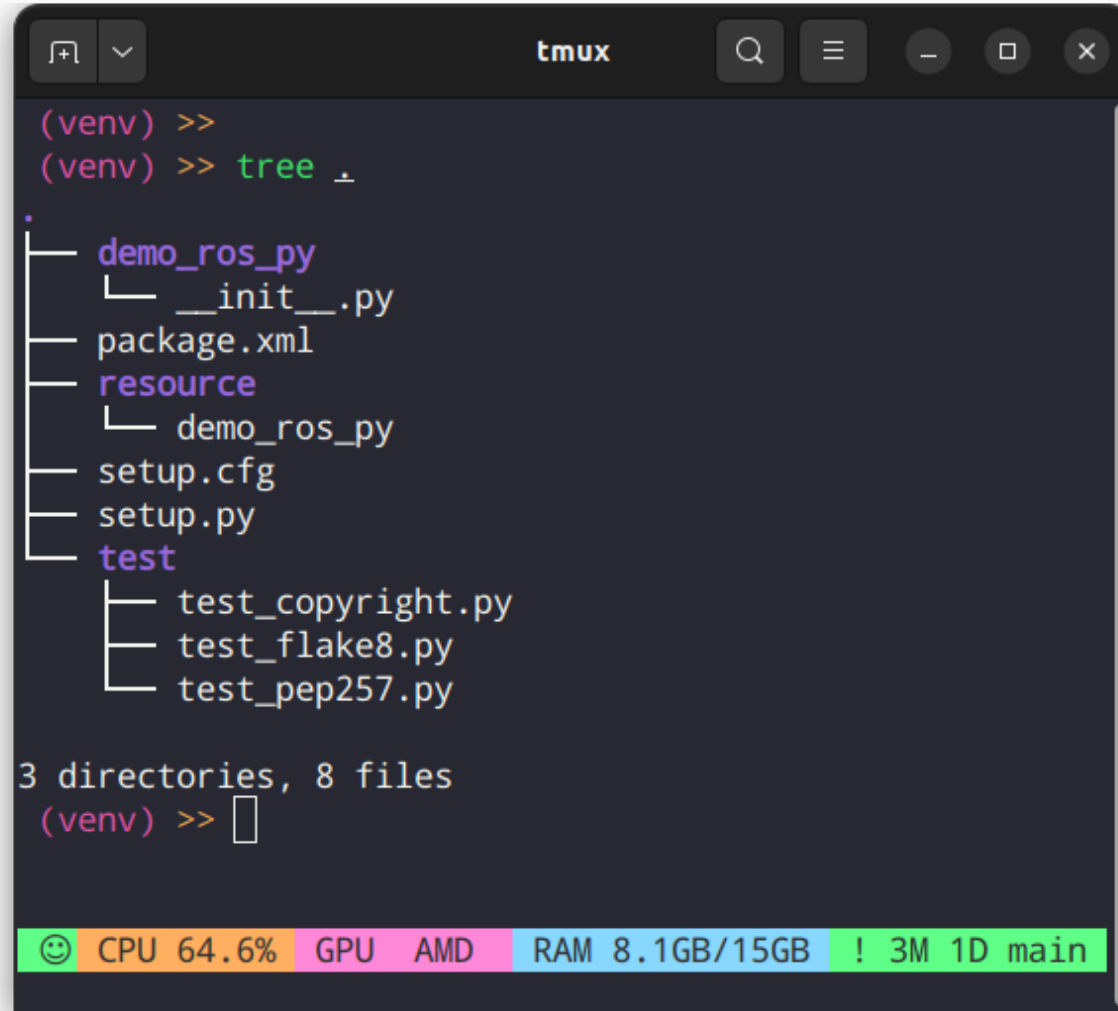


This command creates a new **ROS 2** package with the specified name, using the `ament_python` build type. The generated package structure will look like this:

2.3 Build mechanism

```
1  <package_name>/
2  |— package.xml
3  |— setup.py
4  |— setup.cfg
5  |— resource/
6  |   └— <package_name>
7  |— test/
8  |   |— test_copyright.py
9  |   |— test_flake8.py
10 |   └— test_pep257.py
11 └— <package_name>/
12   └— __init__.py
```


2.3 Build mechanism



A terminal window titled 'tmux' showing the output of the 'tree' command in a virtual environment '(venv)'. The command executed is '(venv) >> tree .'. The output displays a directory tree for the current directory, which contains a subdirectory 'demo_ros_py', a 'package.xml' file, a 'resource' directory, 'setup.cfg', 'setup.py', and a 'test' directory. The 'demo_ros_py' directory contains an '__init__.py' file. The 'resource' directory contains a 'demo_ros_py' file. The 'test' directory contains three files: 'test_copyright.py', 'test_flake8.py', and 'test_pep257.py'. Below the file tree, it states '3 directories, 8 files'. The prompt '(venv) >>' is followed by a cursor. At the bottom of the terminal, a status bar shows system metrics: CPU 64.6%, GPU AMD, RAM 8.1GB/15GB, and network status ! 3M 1D main.

```
(venv) >>
(venv) >> tree .
.
├── demo_ros_py
│   └── __init__.py
├── package.xml
├── resource
│   └── demo_ros_py
├── setup.cfg
├── setup.py
└── test
    ├── test_copyright.py
    ├── test_flake8.py
    └── test_pep257.py

3 directories, 8 files
(venv) >> 
```

😊 CPU 64.6% GPU AMD RAM 8.1GB/15GB ! 3M 1D main

2.3 Build mechanism

i Info

Root Level Files

package.xml The package manifest file containing metadata about the package (*dependencies, version, description, maintainer info, etc.*)

setup.py Python setup script that defines how the package should be built and installed

setup.cfg Configuration file for setup tools, typically contains console script entry points

2.3 Build mechanism

The `package.xml` file is a package manifest for **ROS 2** that describes the package. It's written in **XML** and includes key information like:

- **Metadata:** The package's name, version, a description, maintainer information, and license.
- **Dependencies:** Other packages required for the current package to build and run.
- **Build System Info:** Details on the build type, such as `ament_python`.
- **Export Tags:** Extra information for the **ROS 2** build system.

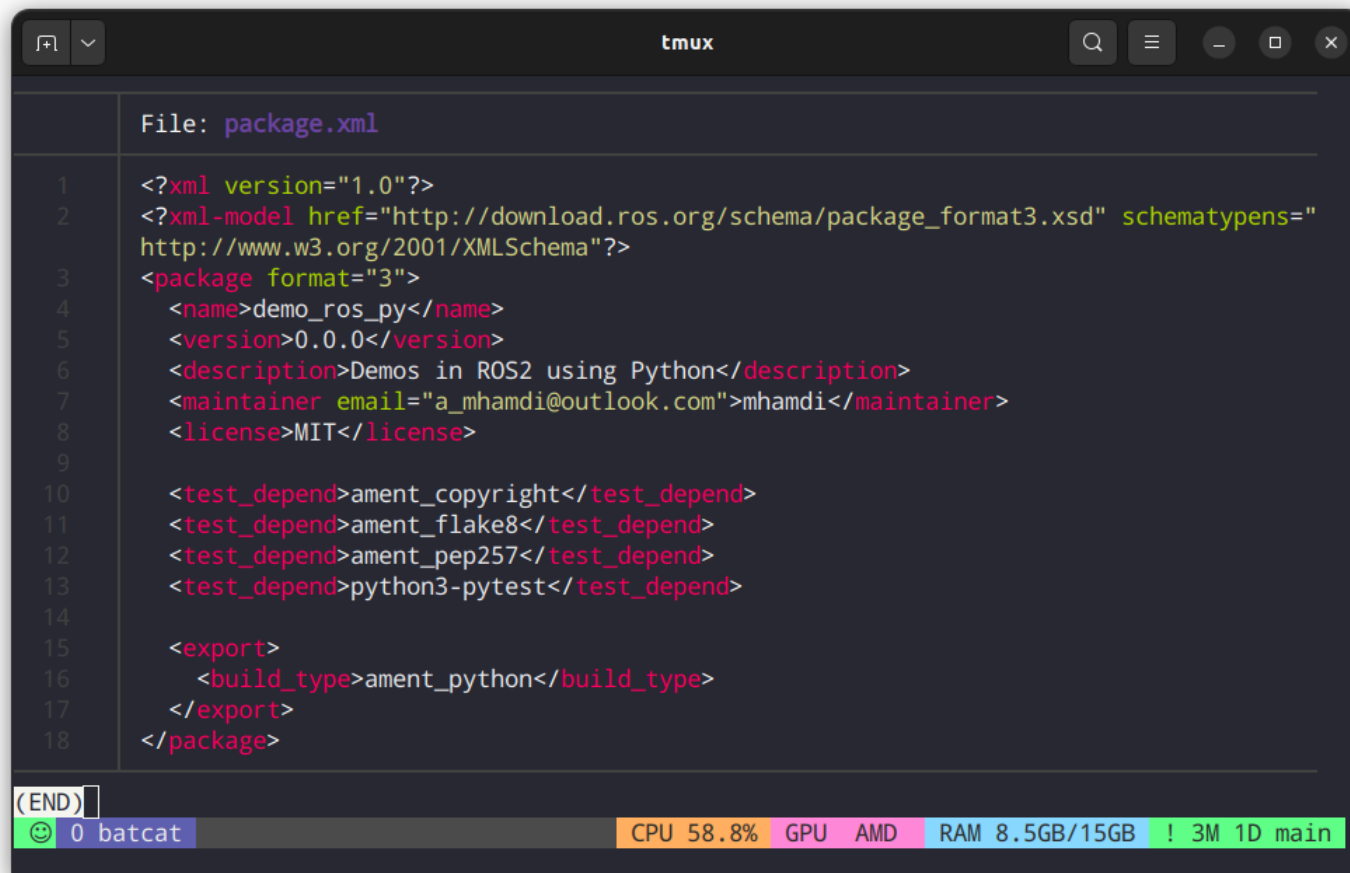
Essentially, this file is how **ROS 2** manages dependencies and compiles our package.

To install the required dependencies, we need to navigate to the package directory and run:

```
1 rosdep install -i --from-path src/<package_name> --rosdistro  
humble -y
```



2.3 Build mechanism



The image shows a tmux terminal window with a dark theme. The title bar at the top says "tmux". The window displays the content of a file named "package.xml". The XML content is as follows:

```
1 <?xml version="1.0"?>
2 <?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="
  http://www.w3.org/2001/XMLSchema"?>
3 <package format="3">
4   <name>demo_ros_py</name>
5   <version>0.0.0</version>
6   <description>Demos in ROS2 using Python</description>
7   <maintainer email="a_mhamdi@outlook.com">mhamdi</maintainer>
8   <license>MIT</license>
9
10  <test_depend>ament_copyright</test_depend>
11  <test_depend>ament_flake8</test_depend>
12  <test_depend>ament_pep257</test_depend>
13  <test_depend>python3-pytest</test_depend>
14
15  <export>
16    <build_type>ament_python</build_type>
17  </export>
18 </package>
```

At the bottom of the terminal, there is a status bar. On the left, it says "(END)". In the middle, it shows system statistics: "0 batcat", "CPU 58.8%", "GPU AMD", "RAM 8.5GB/15GB", and "! 3M 1D main".

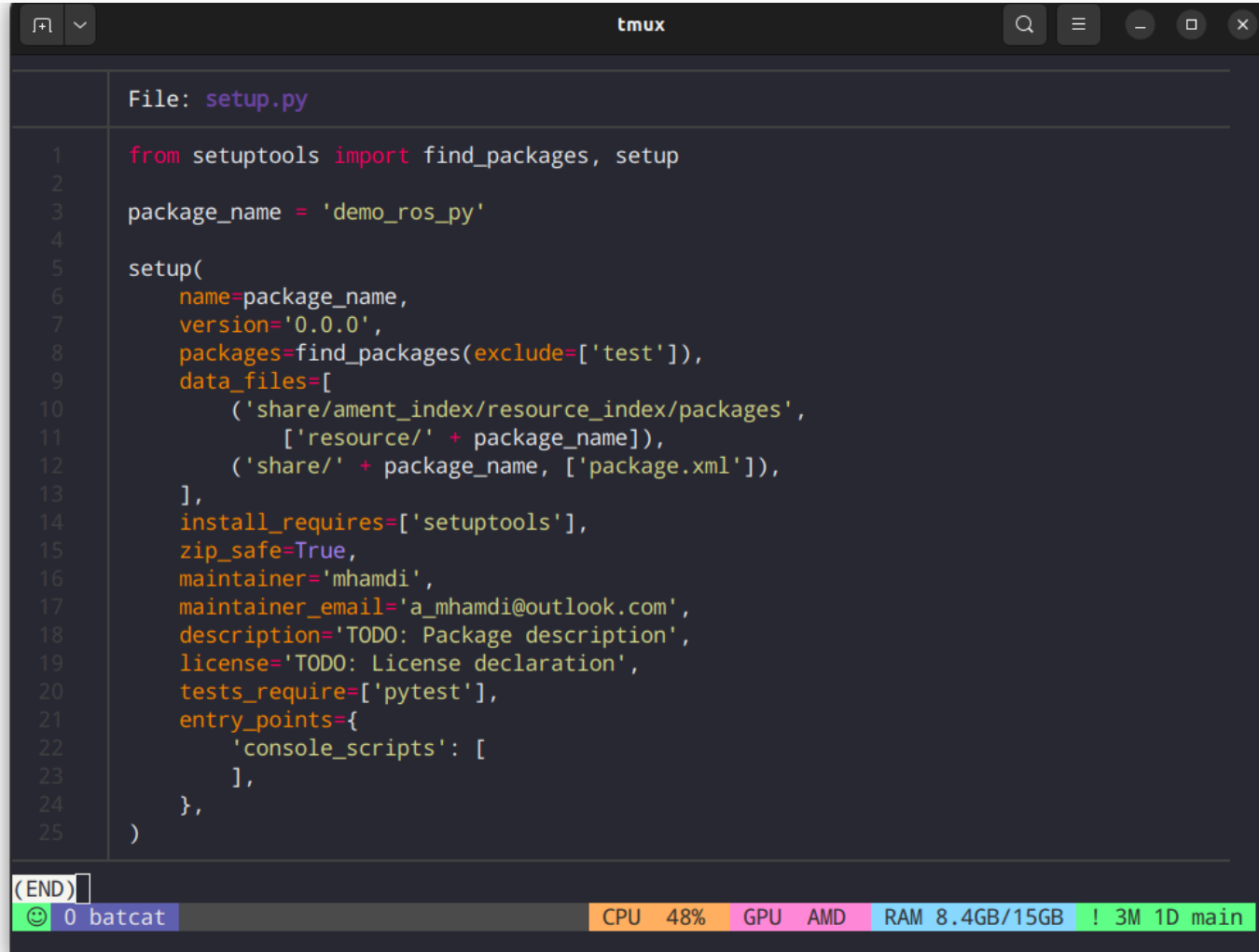
2.3 Build mechanism

The `setup.py` file is a **Python** script that provides instructions for installing a **ROS 2** package. It includes:

- **Metadata:** Package details such as the name, version, and author, which are often sourced from `package.xml`.
- **Dependencies:** The required Python packages for the project.
- **Entry Points:** Specifies console scripts that define **ROS 2** nodes, allowing them to be run as executable commands.
- **Data Files:** Information on any extra files, like launch files or configurations, that need to be installed.
- **Package Discovery:** Instructions for `setuptools` on which **Python** packages to include.

This file uses a standard **Python** packaging mechanism to work with the `ament` build system, making it possible to install and run our **Python** nodes as **ROS 2** executables.

2.3 Build mechanism



```
File: setup.py
1  from setuptools import find_packages, setup
2
3  package_name = 'demo_ros_py'
4
5  setup(
6      name=package_name,
7      version='0.0.0',
8      packages=find_packages(exclude=['test']),
9      data_files=[
10         ('share/ament_index/resource_index/packages',
11          ['resource/' + package_name]),
12         ('share/' + package_name, ['package.xml']),
13     ],
14     install_requires=['setuptools'],
15     zip_safe=True,
16     maintainer='mhamdi',
17     maintainer_email='a_mhamdi@outlook.com',
18     description='TODO: Package description',
19     license='TODO: License declaration',
20     tests_require=['pytest'],
21     entry_points={
22         'console_scripts': [
23         ],
24     },
25 )
```

(END)

🌞 0 batcat CPU 48% GPU AMD RAM 8.4GB/15GB ! 3M 1D main

2.3 Build mechanism

i Info

Directories

resource/<package_name> Contains a marker file (*usually empty*) that helps **ROS 2** identify this as a package

test/ Contains basic test files:

test_copyright.py Checks for proper copyright headers

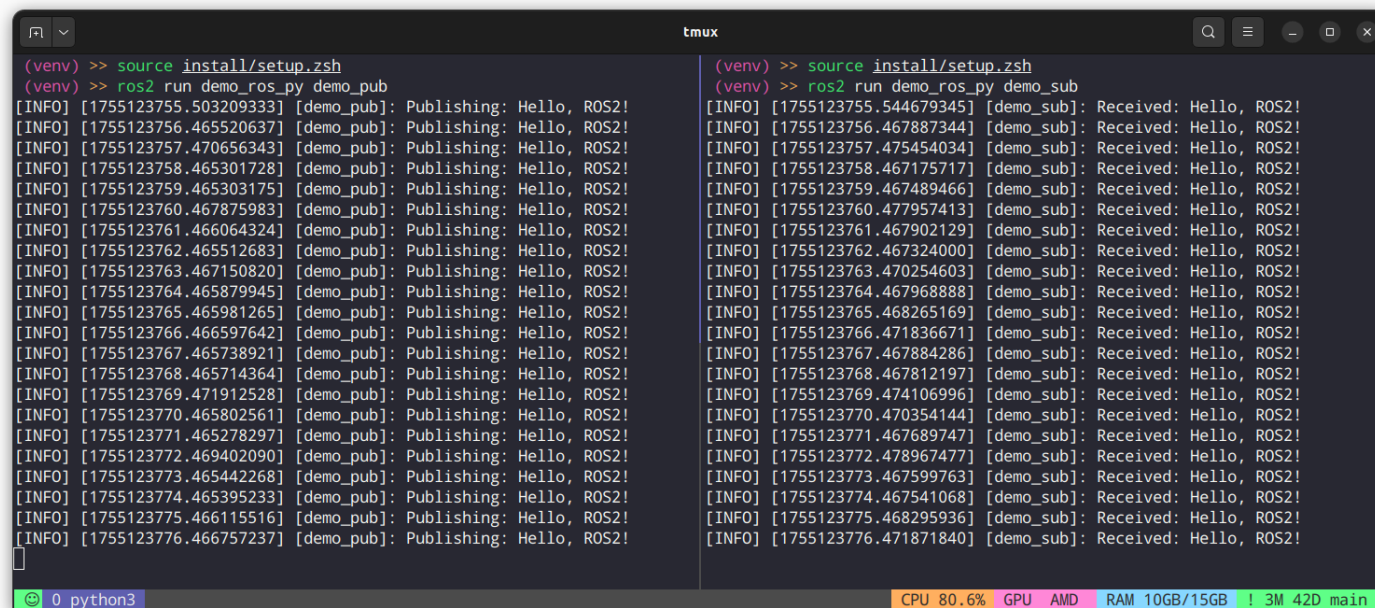
test_flake8.py Runs flake8 linting

test_pep257.py Checks docstring conventions

<package_name>/ The main Python module directory where we'll write our actual **Python** code

__init__.py Makes this directory a Python package

2.3 Build mechanism



```
(venv) >> source install/setup.zsh
(venv) >> ros2 run demo_ros_py demo_pub
[INFO] [1755123755.503209333] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123756.465520637] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123757.470656343] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123758.465301728] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123759.465303175] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123760.467875983] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123761.466064324] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123762.465512683] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123763.467150820] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123764.465879945] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123765.465981265] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123766.466597642] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123767.465738921] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123768.465714364] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123769.471912528] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123770.465802561] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123771.465278297] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123772.469402090] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123773.465442268] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123774.465395233] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123775.466115516] [demo_pub]: Publishing: Hello, ROS2!
[INFO] [1755123776.466757237] [demo_pub]: Publishing: Hello, ROS2!

(venv) >> source install/setup.zsh
(venv) >> ros2 run demo_ros_py demo_sub
[INFO] [1755123755.544679345] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123756.467887344] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123757.475454034] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123758.467175717] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123759.467489466] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123760.477957413] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123761.467902129] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123762.467324000] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123763.470254603] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123764.467968888] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123765.468265169] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123766.471836671] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123767.467884286] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123768.467812197] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123769.474106996] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123770.470354144] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123771.467689747] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123772.478967477] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123773.467599763] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123774.467541068] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123775.468295936] [demo_sub]: Received: Hello, ROS2!
[INFO] [1755123776.471871840] [demo_sub]: Received: Hello, ROS2!
```

0 python3 CPU 80.6% GPU AMD RAM 10GB/15GB ! 3M 42D main



demo_ros_py

ros2_ws/src/demo_ros_py

2.3 Build mechanism

Task 6: Mini Code (Python, minimal)

7. Fill in the blanks to create a simple **ROS 2** publisher node in Python:

```
1  import rclpy
2  from rclpy.node import ____
3  from std_msgs.msg import ____
4
5  class MinimalPublisher(____):
6      def __init__(____):
7          super().__init__('minimal_publisher')
8          self.publisher_ = self.create_publisher(String, '____', 10)
9          timer_period = ____ # seconds
10         self.timer = self.create_timer(timer_period, self.____)
```

 Python

2.3 Build mechanism

```
11
12     def __init__(self):
13         msg = Message()
14         msg.data = ' '
15         self.publisher_ = rospy.Publisher(' ', Message, queue_size=10)
16         self.get_logger().info(f'Publishing: "{msg.data}"')
17
18     def main(args=None):
19         rospy.init_node('rospy_demo', args=args)
20         __ = MinimalPublisher()
21         rospy.spin(__)
22         __.destroy_node()
23         rospy.shutdown()
```

2.3 Build mechanism

```
24  
25 if __name__ == '__main__':  
26     main()
```

3. Custom Interface Development

3. Custom Interface Development

ROS 2 supports three primary types of interfaces for inter-node communication: messages, services, and actions. Each serves a unique purpose in the publish-subscribe or client-server paradigms.

3.2 Messages (msg)

- **Publish-Subscribe** communication
- One-way data transmission
- Asynchronous communication
- Best for: sensor data, status updates, continuous streams

```
1 # Name: "geometry_msgs/msg/Twist.msg"
2 geometry_msgs/Vector3 linear
3 geometry_msgs/Vector3 angular
```

YAML

3.3 Services (srv)

- **Request-Response** communication
- Synchronous, blocking calls
- Best for: commands, queries, configuration

```
1 # Name: "turtlesim/srv/Spawn.srv"
2 float64 x
3 float64 y
4 float64 theta
5 string name
6 ---
7 string name
```

YAML

3.4 Actions (action)

- **Long-running tasks** with feedback
- Asynchronous with progress updates
- Can be cancelled
- Best for: navigation, manipulation, complex behaviors

```
1 # Name: "turtlesim/action/RotateAbsolute.action"
2 float32 theta
3 ---
4 float32 delta
5 ---
6 float32 remaining
```

YAML

3.6 Basic Interface Commands

```
1 ros2 interface <command>
```



```
1 Commands:
```

```
2 list      List all interface types available
```

```
3 package   Output a list of available interface types within one package
```

```
4 packages  Output a list of packages that provide interfaces
```

```
5 proto     Output an interface prototype
```

```
6 show      Output the interface definition
```

3.6 Basic Interface Commands

```
1 ros2 interface package turtlesim
```



```
1 turtlesim/msg/Color
```

```
2 turtlesim/srv/Spawn
```

```
3 turtlesim/msg/Pose
```

```
4 turtlesim/srv/SetPen
```

```
5 turtlesim/srv/Kill
```

```
6 turtlesim/srv/TeleportAbsolute
```

```
7 turtlesim/action/RotateAbsolute
```

```
8 turtlesim/srv/TeleportRelative
```

3.8 Package Structure for Custom Interfaces

```
1 my_interfaces/  
2 |— CMakeLists.txt  
3 |— package.xml  
4 |— msg/  
5 |   └─ SensorData.msg  
6 |— srv/  
7 |   └─ GetData.srv  
8 └─ action/  
9     └─ NavigateTo.action
```

3.8 Package Structure for Custom Interfaces

Key Requirements:

- Separate directories for each interface type
- Proper CMakeLists.txt configuration
- Correct package.xml dependencies

3.9 Message Definition (.msg)

```
1  # Name: "my_interfaces/msg/SensorData.msg"
2  # Header with timestamp and frame
3  std_msgs/Header header
4  # Sensor readings
5  float64 temperature
6  float64 humidity
7  float64 pressure
8  # Status information
9  bool is_valid
10 string sensor_id
```

YAML

3.9 Message Definition (.msg)

Supported Data Types:

- Built-in: `bool`, `int8`, `uint8`, `int16`, `uint16`, `int32`, `uint32`, `int64`, `uint64`, `float32`, `float64`, `string`
- Arrays: `int32[]`, `string[]`
- Other messages: `geometry_msgs/Point`

3.10 Service Definition (.srv)

```
1 # Name: "my_interfaces/srv/GetData.srv"
2 # Request
3 string sensor_id
4 float64 timeout_seconds
5 ---
6 # Response
7 bool success
8 string message
```

YAML

3.10 Service Definition (.srv)

Structure:

- Above ---: Request fields
- Below ---: Response fields
- Can use custom message types

3.11 Action Definition (.action)

```
1  # Name: "my_interfaces/action/NavigateTo.action")
2  # Goal
3  geometry_msgs/Point target_position
4  float64 max_velocity
5
6
7  ---
8  # Result
9  bool success
10 string message
11 float64 final_distance
12
13  ---
```



3.11 Action Definition (.action)

```
14 # Feedback
15 geometry_msgs/Point current_position
16 float64 progress_percentage
17 float64 remaining_distance
```

3.11 Action Definition (.action)

Three Parts:

- **Goal:** What to accomplish
- **Result:** Final outcome
- **Feedback:** Progress updates

3.13 Essential CMakeLists.txt Setup



```
1  # Find dependencies
2  find_package(rosidl_default_generators REQUIRED)
3  find_package(builtin_interfaces REQUIRED)
4  find_package(geometry_msgs REQUIRED)
5  find_package(std_msgs REQUIRED)
6
7  # Generate interfaces
8  rosidl_generate_interfaces(${PROJECT_NAME}
9    "msg/SensorData.msg"
10   "srv/GetData.srv"
11   "action/NavigateTo.action"
12   DEPENDENCIES builtin_interfaces geometry_msgs std_msgs
13   )
```

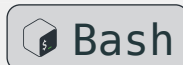
3.14 Package.xml Dependencies

```
1 <name>my_interfaces</name>
2 <version>0.1.0</version>
3 <description>Custom ROS 2 interfaces</description>
4
5 <depend>builtin_interfaces</depend>
6 <depend>geometry_msgs</depend>
7 <depend>std_msgs</depend>
```



3.16 Building the Package

```
1 # Build the interface package
2 colcon build --packages-select my_interfaces
3
4 # Source the workspace
5 source install/setup.bash
```



3.16 Building the Package

Verification:

```
1 # List available interfaces
2 ros2 interface list | grep my_interface_package
3
4 # Show interface definition
5 ros2 interface show my_interfaces/msg/SensorData
6
7 # Check package interfaces
8 ros2 interface package my_interfaces
```



3.17 Using Custom Interfaces in Python and C++

```
1 from my_interfaces.msg import SensorData
2 from my_interfaces.srv import GetData
3 from my_interfaces.action import NavigateTo
```

 Python

```
1 #include "my_interfaces/msg/sensor_data.hpp"
2 #include "my_interfaces/srv/get_data.hpp"
3 #include "my_interfaces/action/navigate_to.hpp"
```

 C++

3.17 Using Custom Interfaces in Python and C++

```
1 # Publisher
2 publisher = node.create_publisher(SensorData, 'sensor_data', 10)
```

 Python

```
1 // Publisher
2 auto publisher = node->create_publisher<my_interfaces::msg::SensorData>(
3     "sensor_data", 10);
```

 C++

3.17 Using Custom Interfaces in Python and C++

```
1 # Service client
2 client = node.create_client(GetData, 'get_data')
```

 Python

```
1 // Service client
2 auto client = node->create_client<my_interfaces::srv::GetData>(
3     "get_data");
```

 C++

3.17 Using Custom Interfaces in Python and C++

```
1 # Action client
```

 Python

```
2 action_client = ActionClient(node, NavigateTo, 'navigate_to')
```

```
1 // Action client
```

 C++

```
2 auto action_client =  
  rclcpp_action::create_client<my_interfaces::action::NavigateTo>(  
3   node, "navigate_to");
```

3.19 Interface Validation



```
1  # Validate message before publishing
2  from rclpy.qos import QoSProfile, ReliabilityPolicy
3
4  # Set up validation
5  qos = QoSProfile(
6      reliability=ReliabilityPolicy.RELIABLE,
7      depth=10
8  )
9
10 # Use with validation
11 publisher = node.create_publisher(
12     SensorData, 'sensor_data', qos_profile=qos
13 )
```

3.20 Custom Interface Testing

```
1 # Test interface compilation
2 colcon build --packages-select my_interfaces
3
4 # Test interface usage
5 ros2 run my_interfaces test_interfaces
6
7 # Validate with rosbag
8 ros2 bag record /sensor_data
9 ros2 bag play recorded_bag
```



Testing Strategies:

- Unit tests for interface generation
- Integration tests for message flow
- Performance tests for large messages

3.21 Summary

Interface Types

- Messages: One-way, asynchronous
- Services: Request-response, synchronous
- Actions: Long-running with feedback

Development Process

1. Define interface files (.msg, .srv, .action)
2. Configure CMakeLists.txt and package.xml
3. Build with colcon

Best Practices

- Design for reusability
- Maintain backward compatibility
- Test thoroughly
- Document clearly

Thank you for your attention

Bibliography

Bibliography

- Brito, B., Marques, H., Oso, A., Camacho, A., Olivares-Alarcos, A., Foix, S., Perera, A., Porzi, L., Santos, C., & Kappler, D. (2021). ROS2Learn: A reinforcement learning framework for ROS 2. *Journal of Intelligent & Robotic Systems*, 102(4), 1–19.
- Castelló, J., Macenski, S., & Martín, F. (2021). Navigation2: A new generation of navigation for robots using ROS 2. *IEEE Robotics & Automation Magazine*, 28(4), 87–98.
- Di Nardo, D., Cicconetti, C., Giambene, G., Muhammad, T., Spada, F., & Bechini, A. (2022). Security challenges in Robot Operating System 2 (ROS 2): An empirical analysis. 2022 *IEEE 23rd International Symposium on a World of Wireless, Mobile and Multimedia Networks (Wowmom)*, 466–472.
- Hernández, E., Pérez, V., Martín, F., & Fernández, J. (2021). ROS 2 for robotics applications: A comprehensive review. *Sensors*, 21(24), 8240.
- Joseph, L., & Cacace, J. (2018). *Mastering ROS for Robotics Programming: Design, build, and simulate complex robots using the Robot Operating System*. Packt Publishing Ltd.

- Koubaa, A. (2018). Service-oriented Robot Operating System for distributed robotics: A case study. *Robotics and Autonomous Systems*, 108, 91–109.
- Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), eabm6074.
- Maruyama, Y., Kato, S., & Azumi, T. (2016). Exploring the performance of ROS2. *Proceedings of the 13th International Conference on Embedded Software*, 1–10.
- Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., & Ng, A. Y. (2009). ROS: an open-source Robot Operating System. *ICRA Workshop on Open Source Software*, 3(3.2), 5.
- Reke, M., Peter, D., Schulte-Tigges, J., Schiffer, S., Ferrein, A., Walter, T., & Matheis, D. (2020). Real-time ROS 2 communication for cooperative automated vehicles. *2020 IEEE Intelligent Vehicles Symposium (IV)*, 958–963.